

Landscape Analysis of Temporal-Aware Colonoscopy Polyp Detection Systems: Strategic Blueprint for the ChakraModel Prototype

Executive Summary

The transition of computer-aided polyp detection (CAPD) systems from controlled academic environments into live clinical application is currently obstructed by a critical operational failure: temporal instability. Colorectal cancer (CRC) remains the third most frequent cancer globally and a leading cause of cancer-related mortality, primarily developing through a well-documented adenoma-carcinoma sequence where benign polyps progressively transform into malignancies¹. While colonoscopy is the gold standard for intercepting this progression, its efficacy is severely limited by human performance factors, operator fatigue, and challenging lighting conditions, resulting in an alarming polyp miss rate of 14% to 30%¹. Artificial intelligence models have demonstrated exceptional theoretical promise in mitigating this issue, yet existing deep learning architectures frequently suffer from high variance when deployed on continuous video streams. In a live colonoscopy, rapid camera movements, water washout, specular reflections from mucosal fluids, and partial occlusions by haustral folds or fecal matter cause frame-to-frame detection flickering. A polyp identified in one frame may lose its bounding box in the next, a phenomenon that induces clinician cognitive fatigue, erodes trust in the diagnostic tool, and ultimately contributes to missed lesions.

The initial proposition of the ChakraModel conceptualizes a hybrid architecture utilizing YOLOv8 for primary fast proposals and a Region-based Convolutional Neural Network (R-CNN) for artifact-aware refinement. While this dual-stage pipeline theoretically addresses localization robustness, an exhaustive analysis of the current state of the art—evaluated through the strict constraints of a 36-hour development cycle by a three-person team with limited machine learning expertise—reveals significant feasibility barriers. Building, integrating, and optimizing a live inference pipeline that passes high-speed YOLO tensor proposals into a heavy R-CNN refinement stage, while maintaining the mandatory real-time latency of at least 25 frames per second (FPS), requires complex CUDA-level memory management and custom PyTorch engineering that cannot be realistically accomplished within a hackathon window³.

Consequently, this investigation dictates a strategic pivot to guarantee the delivery of a clinically viable prototype by August 1. The most actionable path to achieving a temporally consistent, artifact-aware detection pipeline involves deprecating the custom R-CNN refinement stage. Instead, the architecture must leverage the native, highly optimized tracking capabilities of the Ultralytics YOLO API—specifically utilizing algorithms like ByteTrack—layered with a custom temporal persistence logic module designed to act as short-term memory⁵. This single-stage,

tracker-augmented pipeline can be rapidly deployed on Google Colab GPUs, achieves superior artifact suppression, maintains high inference speeds, and allows the biomedical engineering domain expertise of the team to focus on high-impact data augmentation and the implementation of custom evaluation metrics: the Temporal Consistency Score (TCS) and Artifact Rejection Score (ARS).

Existing Approaches and Implementation Feasibility

The landscape of automated polyp detection is heavily dominated by Convolutional Neural Networks (CNNs) and is currently undergoing a paradigm shift toward real-time, single-shot architectures and specialized video-processing networks. The following approaches have been rigorously evaluated based on their relevance to the ChakraModel objectives, their baseline performance metrics, and crucially, their implementation speed for a 36-hour development window.

1. Ultralytics YOLO Architectures (v8 and v11)

The YOLO (You Only Look Once) architecture represents the industry standard for real-time object detection, achieving a dominant balance between inference speed and bounding box precision. Recent iterations, particularly YOLOv8 and YOLOv11, have introduced significant structural enhancements that are highly relevant to medical imaging. YOLOv8 utilizes an anchor-free split Ultralytics head and state-of-the-art Cross-Stage Partial (CSP) bottleneck architectures, specifically the C2f modules⁵. In the context of colonoscopy, where polyps present in highly variable shapes (sessile, pedunculated, or flat) and sizes, the anchor-free design prevents the model from being biased toward predefined bounding box ratios¹. Furthermore, the integration of a self-attention mechanism within the network allows the model to capture complex spatial dependencies within the colonic mucosa, aiding in differentiating polyps from surrounding normal tissue or artifacts⁵.

Recent benchmarking on colonoscopy datasets indicates that YOLO-based models are unparalleled in real-time environments. YOLOv11 has demonstrated the ability to achieve up to 99.37% mAP@0.5 and 97.64% Intersection over Union (IoU) on the Kvasir-SEG dataset under optimal, region-guided training conditions⁹. Even standard YOLOv8 weights yield F1-scores exceeding 92% on combined static datasets while maintaining processing speeds well above 50 FPS on standard hardware¹⁰.

For a rapid prototyping environment, the Ultralytics framework offers the highest leverage. The Python API is plug-and-play, installable via a simple pip command, and abstracts away the complexities of PyTorch training loops⁵. Pretrained weights on the COCO dataset (yolov8n.pt through yolov8x.pt) are publicly accessible and download automatically upon initialization, serving as a highly effective foundation for transfer learning¹¹. Most importantly for the ChakraModel's temporal consistency goals, the YOLO API natively supports multi-object tracking via ByteTrack and BoT-SORT by simply passing a `tracker="bytetrack.yaml"` argument during inference⁵. This allows the team to implement sophisticated temporal tracking in minutes rather than days.

2. Adaptive Video Polyp Detection Network (AVPDN)

AVPDN is a state-of-the-art framework explicitly designed to address the exact clinical problems ChakraModel targets: motion blur, specular reflections, and severe scale variations caused by rapid camera movement in dynamic colonoscopy frames¹³. Built upon the real-time RT-DETR detection framework, AVPDN introduces highly specialized architectural components. The Adaptive Feature Interaction and Augmentation (AFIA) module utilizes a dual-branch self-attention architecture that perfectly balances global context modeling (via Dense Self-Attention) with noise suppression (via Sparse Self-Attention), utilizing channel shuffle operations to enhance inter-branch information exchange¹³. This mechanism is specifically engineered to suppress task-irrelevant noise, such as water flushes and glaring lights. Concurrently, a Scale-Aware Context Integration (SACI) module leverages dilated convolutions with varying receptive fields to capture the spatial distribution of polyps as the endoscope rapidly approaches or withdraws from the lesion¹⁴.

AVPDN's performance on dynamic video sequences is exceptional. On the highly challenging CVC-VideoClinicDB dataset, AVPDN achieved an Average Precision (AP) of 95.3% while operating at 53.2 FPS, outperforming standard baseline models like YOLOv8 and Faster R-CNN in temporal environments¹³.

The implementation feasibility for a 36-hour hackathon, however, carries moderate risk. The code and pretrained weights are fully open-source and available on GitHub (xiaochen925/AVPDN) without access restrictions¹³. However, the repository requires the configuration of a specific Conda virtual environment, adherence to rigid YAML configuration files, and the preparation of data in a highly specific directory structure¹³. For a team with limited deep learning engineering experience, adapting this custom PyTorch codebase, debugging dependency conflicts, and modifying dataloaders for custom evaluation metrics could consume the entirety of the hackathon, leaving no time for clinical validation or UI development.

3. Faster R-CNN with Temporal Exploitation

Early academic attempts to introduce temporal consistency into colonoscopy analysis frequently utilized two-stage detectors like Faster R-CNN. Researchers, notably Qadir et al., recognized that single-frame detectors were easily fooled by circular bubbles or mucosal folds. They proposed integrating a false-positive reduction module that explicitly exploited temporal dependencies between consecutive video frames³. By tracking regions of interest (ROIs) and applying temporal attention across a sliding window of frames, these models successfully suppressed isolated false positives caused by transient artifacts, ensuring that a detection was only finalized if the feature representations remained consistent over time¹⁶.

While this approach yields high precision and specificity, it is fundamentally incompatible with the constraints of real-time clinical deployment and rapid prototyping. Two-stage detectors inherently suffer from latency issues; generating region proposals before passing them to a classification head often drops inference speeds below the required 25 FPS threshold unless deployed on massive, data-center-grade GPUs³. Furthermore, while generic Faster R-CNN implementations exist within torchvision, the specific temporal ROI alignment modules and 3D

attention mechanisms designed in these studies are not readily available as plug-and-play libraries¹⁵. Implementing a custom temporal ROI alignment mechanism over a two-stage detector demands extensive low-level tensor manipulation and debugging, rendering it entirely unfeasible for a 36-hour sprint.

4. ECC-PolypDet and Contrastive Learning Frameworks

Another sophisticated approach to handling artifact robustness involves optimizing the latent space of the model rather than altering the temporal inference pipeline. ECC-PolypDet (Enhanced CenterNet with Contrastive Learning) utilizes Box-assisted Contrastive Learning (BCL) during the training phase to minimize intra-class differences (making all polyps look mathematically similar in the feature space) and maximize inter-class differences between foreground polyps and the highly variable background mucosa¹⁸. This makes the model highly robust against concealed polyps and complex artifacts like fecal matter.

While the mathematical foundation of this approach is highly relevant to suppressing artifact-induced false alarms, its practical implementation is obstructed by several barriers. The GitHub repository (yuncheng97/ECC-PolypDet) hosts its pretrained weights exclusively on Baidu Drive, which frequently presents accessibility and bandwidth issues for international researchers¹⁸. Additionally, integrating contrastive loss functions into a training pipeline requires meticulous tuning of hyperparameters and batch sizes, a process that is too time-consuming and fragile for a rapid hackathon environment.

Framework	Core Architecture	Key Innovation	Benchmark Performance	Implementation Feasibility	Integration Time Estimate
Ultralytics YOLOv8/v11	Single-stage, anchor-free CNN	Real-time speed, native ByteTrack integration	>90% mAP, >50 FPS on static datasets ⁹	High. Plug-and-play API, public weights ⁵ .	1 - 3 hours
AVPDN	RT-DETR variant	Dual-branch self-attention for motion blur	95.3% AP, 53.2 FPS on VideoClinic DB ¹³	Moderate. Requires custom Conda setup and YAML tuning ¹³ .	6 - 10 hours
Temporal Faster	Two-stage CNN with	Temporal attention	High precision,	Low. Requires	> 48 hours

R-CNN	ROI tracking	aggregation across frames	but low FPS (<20) ³	custom PyTorch engineering ¹⁷ .	
ECC-Polyp Det	CenterNet	Box-assisted contrastive learning	High robustness to concealed polyps ¹⁸	Low. Baidu Drive dependencies, complex training pipeline ¹⁸ .	> 24 hours

Why Progress Stalled: Bottlenecks to Clinical Adoption

Despite the publication of thousands of papers on deep learning for polyp detection, the transition from academic novelty to standard-of-care clinical tools remains sluggish. An analysis of medical AI literature and practitioner discussions reveals several systemic bottlenecks that directly inform the strategic decisions required for the ChakraModel prototype.

The Static Dataset Fallacy and Domain Shift

The vast majority of computer-aided polyp detection models are trained, validated, and benchmarked exclusively on static image datasets, such as Kvasir-SEG and CVC-ClinicDB²⁰. These datasets are highly curated; they consist of clear, well-illuminated frames where polyps are generally centered and easily distinguishable from the background²². When models mathematically optimized for these pristine conditions are deployed on live endoscopic video streams, they encounter a severe "domain shift." Dynamic colonoscopy is chaotic, characterized by motion blur from rapid scope withdrawal, liquid flushes to clean the lens, and harsh specular reflections from mucosal fluids¹³. Algorithms that achieve 95% mAP on Kvasir-SEG routinely see their performance degrade precipitously when faced with these unrepresented dynamic artifacts, resulting in a deluge of false positives and missed detections.

The Psychological Impact of the Flickering Phenomenon

In real-world applications, a lack of temporal consistency results in detection bounding boxes flashing on and off rapidly. If a polyp is briefly obscured by a water flush for 200 milliseconds, a purely frame-wise detector will drop the bounding box and then redraw it a fraction of a second later. This flickering is disastrous in a clinical setting. It drastically increases the cognitive load on the gastroenterologist, causing visual fatigue, irritation, and distraction from the actual procedure²³. In many cases, clinicians find the flickering so disruptive that they simply ignore the CAD system or turn it off entirely, negating any theoretical benefits of the AI. Clinical trust is predicated on stability; a system that holds a steady bounding box through transient artifacts is infinitely more valuable than one that is hyper-accurate but visually erratic.

Computational Constraints of Hospital Hardware

Many academic solutions attempt to solve the temporal instability problem by employing 3D Fully Convolutional Networks (3D-FCNs), Recurrent Neural Networks (RNNs), or complex Spatio-Temporal Vision Transformers²⁴. While these architectures are highly effective at modeling time and sequence, they are computationally exorbitant. Operating rooms utilize standard endoscopy towers (e.g., Olympus, Pentax) equipped with modest, embedded hardware systems. A deep learning solution that requires multiple high-end NVIDIA GPUs to achieve real-time latency is practically useless for mass deployment across rural or underfunded hospital networks⁴. The industry demands lightweight architectures that can process high-definition video at 25 to 30 FPS on edge devices.

The Inherent Complexity of Hybrid Pipelines

The original ChakraModel prompt suggests a hybrid approach: YOLO for proposals and R-CNN for refinement. Historically, ensembling a single-stage detector with a two-stage detector results in catastrophic latency accumulation. If a polyp bounding box generated by YOLO takes 15ms to compute, and passing that crop to an R-CNN feature extractor takes an additional 40ms, the total pipeline latency pushes the system well below the 25 FPS threshold required for live, stutter-free colonoscopy video³. For a hackathon team, attempting to manage the CUDA memory handoffs between two distinct neural network architectures will guarantee a failure to produce a working prototype.

Alternative Technical Directions

To circumvent the computational bottlenecks of 3D convolutions and the latency traps of hybrid architectures, several alternative technical directions offer pathways to achieving artifact robustness and temporal stability within a 36-hour window.

1. Lightweight Object Tracking (Highly Feasible)

Rather than using a massive neural network to implicitly model time, algorithmic object tracking utilizes statistical physics and geometry to predict where a bounding box will be in the next frame. Algorithms like SORT (Simple Online and Realtime Tracking), DeepSORT, and ByteTrack utilize Kalman filters to predict the future trajectory of a bounding box based on its velocity and acceleration in previous frames⁵. The Hungarian algorithm is then used to match these predictions with the actual neural network detections in the current frame based on Intersection over Union (IoU) metrics.

ByteTrack is particularly relevant for colonoscopy. Traditional trackers discard low-confidence bounding boxes, which causes tracks to break when a polyp is partially obscured by fecal matter. ByteTrack recovers these low-confidence boxes by associating them with existing track trajectories, effectively maintaining a lock on the polyp even when the visual evidence is weak⁵. This approach is incredibly feasible for the hackathon because ByteTrack is integrated directly into the Ultralytics YOLOv8 Python API, requiring almost zero custom engineering to deploy.

2. Synthetic Data Generation via Diffusion Models (Moderate

Feasibility)

Artifact robustness can be achieved through data-centric AI rather than model-centric structural changes. Frameworks like ControlPolypNet utilize text-to-image models (Stable Diffusion combined with ControlNet) to generate synthetic polyps under various simulated clinical conditions, such as extreme lighting angles or artifact occlusions²⁶. By controlling the generation process with custom-masked input images, researchers can artificially inflate the training dataset with clinically relevant "hard cases."

While training or fine-tuning a diffusion model from scratch is impossible in 36 hours, the underlying principle is highly actionable. A biomedical engineer on the team can utilize standard, high-speed augmentation libraries (like Albumentations) to mathematically simulate these artifacts (e.g., adding Gaussian blur to simulate motion, applying Cutout to simulate instrument occlusion, and overlaying synthetic glare) during the YOLO training phase, achieving similar robustness without the generative overhead⁵.

3. Optical Flow Motion Compensation (Not Feasible)

Optical flow algorithms calculate the dense pixel-level displacement between consecutive frames, allowing a model to physically warp the feature maps of a previous frame to match the geometry of the current frame. This can bridge gaps caused by transient artifacts and align temporal data perfectly³. However, calculating dense optical flow (e.g., via neural networks like FlowNet) adds severe computational overhead to the inference pipeline. Furthermore, merging flow features with detection features requires designing and training a custom network architecture from the ground up, making this approach entirely unfeasible for a rapid prototyping scenario.

Available Resources

The success of a 36-hour hackathon relies entirely on the ruthless exploitation of existing, high-quality open-source assets. Attempting ground-up data labeling or model initialization will invariably result in a failure to deliver a working prototype.

Open-Access Datasets

The medical AI community has curated several vital datasets that must be leveraged for training and evaluation.

Dataset	Size and Format	Characteristics	Download & Setup	License
Kvasir-SEG [cite: 2, 27]	1,000 images & binary masks (46.2 MB)	Highly curated static images. Varied resolution (332x487 to	< 5 minutes via Kaggle (debeshjha1/kvasirseg) or HuggingFace	CC-BY 4.0 ²⁸

		1920x1072). Contains JSON bounding boxes.	(MedOtter/kvas ir-seg).	
CVC-ClinicDB [cite: 30, 31, 32]	612 images & binary masks (50 MB)	Static frames extracted from 31 video sequences. Standardized resolution (384x288). Requires mask-to-box conversion.	< 5 minutes via Kaggle (balraj98/cvccli nicdb) or Roboflow Universe.	Research/Educ ational use only (No commercial) ³¹
CVC-VideoCli nicDB [cite: 3]	36 videos, 11,954 frames	Dynamic video sequences recorded at 25 FPS from standard white-light endoscopes.	~30 minutes via Kaggle or specific GitHub repos.	Research use only
PolypGen [cite: 22]	1,537 static images & 2,225 sequence frames	Multi-center dataset (6 distinct clinical centers). Essential for testing domain generalization.	~15 minutes via Supervisely or open academic portals.	CC-BY 4.0 ²²
HyperKvasir [cite: 28]	110,079 images, 373 videos	Massive comprehensive dataset including normal tissue, polyps, and artifacts.	1-2 hours. Large size necessitates selective downloading for a hackathon.	CC-BY 4.0 ²⁸

Integration Note for the Team: Kvasir-SEG and CVC-ClinicDB are formatted as semantic segmentation datasets, meaning they provide a black background with a white foreground mask indicating the polyp³. Object detectors like YOLO require bounding box coordinates in a specific text format (class x_center y_center width height). The team must write a preprocessing script to convert these masks before training can begin. CVC-VideoClinicDB is absolutely critical for the evaluation phase, as it is the only way to accurately calculate the proposed Temporal Consistency Score (TCS) on continuous, artifact-heavy sequences³.

Pretrained Models and Code Repositories

Model / Repository	Source	Benchmark Performance	Integration Effort	Weights Available
Ultralytics YOLO [cite: 8, 11]	github.com/ultralytics/ultralytics	>50 FPS, ~90% mAP (varies by variant size: n, s, m, l, x).	Plug-and-play. 1-2 hours. Extensive documentation and community support.	Yes (yolov8n.pt through yolov8x.pt)
AVPDN [cite: 13]	github.com/xiaochen925/AVPDN	95.3% AP, 53.2 FPS on VideoClinicDB.	Moderate. Requires specific Conda setup and custom PyTorch dataloaders ¹³ .	Yes (via GitHub Release) ¹³
YOLO-Colonoscopy [cite: 33, 34]	github.com/ArdEleanRichard/YOLO-Colonoscopy	Evaluates YOLOv8-v12. 70.2% mAP on specific cross-dataset splits.	Low. Contains pre-configured YAML files for polyp datasets.	Relies on Ultralytics base weights

Technical Opportunities and "Quick Wins"

Given the constraints of a 3-person team with distributed expertise, ChakraModel must exploit specific technical blind spots in the literature that require minimal deep engineering but yield disproportionately high clinical value.

1. Mask-to-Box Automation pipeline

The necessity of converting Kvasir-SEG and CVC-ClinicDB segmentation masks into YOLO bounding box text files presents an immediate bottleneck.

- **The Quick Win:** This task perfectly suits the biomedical engineer or the generalist. A simple Python script utilizing the OpenCV library (`cv2.findContours` followed by `cv2.boundingRect`) can iterate through thousands of masks and generate normalized YOLO coordinate files in under 10 minutes². This bypasses the need to research or deploy a complex segmentation architecture (like U-Net) and allows the ML engineer to immediately begin training a high-speed bounding box detector.

2. Artifact-Aware Data Augmentation

Instead of attempting to build a bespoke R-CNN refinement stage to handle artifacts, the team can force the YOLO model to learn artifact immunity during the primary training phase.

- **The Quick Win:** Utilize the Albumentations library to apply aggressive, clinically relevant visual distortions to the training images. By applying MotionBlur (to simulate rapid scope movement), OpticalDistortion, and CoarseDropout (to simulate fecal matter, bubbles, or instrument occlusion), the dataset becomes artificially hostile¹⁸. This computationally free technique drastically reduces false positives in dynamic video without changing a single line of neural network architecture.

3. The Temporal Persistence Logic Module (The Core Innovation)

To directly solve the clinical "flickering" problem and maximize the proposed Temporal Consistency Score (TCS), the team must implement a short-term memory tracking wrapper.

- **The Quick Win:** Utilize YOLOv8's native, highly optimized tracking API as the base: `results = model.track(source="video.mp4", tracker="bytetrack.yaml", persist=True)`⁵.
- **Persistence Logic Implementation:** The ML engineer should write a custom Python wrapper around this inference loop. Maintain a dictionary named `track_history`. If a specific tracked polyp ID disappears for N consecutive frames (where $N < 5$, representing approximately 200ms of water washout or glare), the logic module artificially sustains the bounding box at its last known coordinates, or extrapolates its trajectory using simple velocity vectors. If the track remains missing beyond N frames, it is deleted⁶. This perfectly smooths the visual output for the clinician, eliminating cognitive fatigue without requiring complex temporal neural networks.

4. Custom Task-Focused Evaluation Metrics

The medical AI literature obsessively optimizes for mAP and IoU on static images, metrics which do not directly translate to clinical trust or usability in a live procedure²¹.

- **The Quick Win:** The team can define, code, and heavily promote two novel evaluation metrics: the Temporal Consistency Score (TCS) and Artifact Rejection Score (ARS).
 - *TCS Formulation:*

$$TCS = 1 - (\text{Number of Track ID switches} / \text{Total True Positive Frames})$$

. A mathematical penalty is applied every time a polyp's bounding box drops and

reappears, quantifying stability.

- *ARS Formulation:*

$$ARS = \text{True Negatives} / (\text{True Negatives} + \text{Artifact-Induced False Positives})$$

. This should be evaluated on specific, known negative video sequences in CVC-VideoClinicDB that contain high amounts of mucosal debris and bubbles, proving the model does not hallucinate polyps³.

Recommended Reading

To rapidly upskill the biomedical engineer and ML engineer, the following papers provide the highest actionable value for the hackathon execution. The team should avoid reading deeply into 3D-CNNs or Transformer architectures and focus strictly on these pragmatic texts:

1. **"AVPDN: Learning Motion-Robust and Scale-Adaptive Representations for Polyp Detection in Dynamic Colonoscopy Frames"** (Chen & Lu, 2025)¹³.
 - *Relevance:* This is the definitive modern text outlining the specific artifacts (motion blur, scaling, specular highlights) found in dynamic colonoscopy¹⁴.
 - *Hackathon Application:* Even if the team does not utilize the complex AVPDN codebase, the paper's rigorous categorization of dynamic artifacts dictates exactly how the biomedical engineer should configure the Albumentations data augmentation pipeline to simulate real-world hostility.
2. **"Real-Time Polyp Detection, Localization and Segmentation in Colonoscopy Using Deep Learning"** (Jha et al., 2021)³⁷.
 - *Relevance:* This paper thoroughly benchmarks standard object detectors (YOLOv4, Faster R-CNN, RetinaNet) on the Kvasir-SEG dataset.
 - *Hackathon Application:* It provides baseline expectations for inference speed (FPS) and accuracy limits. Crucially, it demonstrates that YOLO architectures consistently offer the best speed/accuracy tradeoff, empirically validating the team's strategic decision to drop the R-CNN stage²¹.
3. **"Colorectal Polyp Detection and Localization in Colonoscopy Images Based on YOLOv8"** (Lalinia & Sahafi, 2024)¹.
 - *Relevance:* Specifically details the implementation nuances of YOLOv8 for colorectal polyps, addressing dataset formatting and preprocessing.
 - *Hackathon Application:* Serves as a direct blueprint for training the base ChakraModel, saving the ML engineer hours of trial and error in hyperparameter configuration.
4. **Kvasir-SEG and CVC-ClinicDB Dataset Publications** (Jha et al., 2020; Bernal et al., 2015)²¹.
 - *Relevance:* Outlines the clinical context, the anatomical ground truth of the annotations, and standard evaluation metrics (Dice, IoU)³.
 - *Hackathon Application:* Essential reading for the biomedical engineer to understand the biological variations of polyps (sessile vs. pedunculated) and to handle ethical/licensing compliance statements during the final hackathon presentation.

36-Hour Reality Check and Execution Strategy

A 3-person team (comprising one ML engineer, one biomedical engineer, and one generalist) starting with zero existing code faces severe critical path dependencies. **The original ChakraModel proposal—a YOLOv8 plus R-CNN hybrid—is a fatal scope trap.** Attempting to build a custom pipeline that passes YOLO tensor outputs into a PyTorch `torchvision.models.detection.fasterrcnn` feature extractor, resolving CUDA memory overlaps, and forcing the ensemble to run at >25 FPS is practically impossible in 36 hours. To successfully ship a highly impressive, clinically viable prototype by the August 1 deadline, the team must execute the following pivot and rigid timeline:

Explicit Scope Trade-offs

- **CUT:** The R-CNN refinement stage. Completely abandon two-stage detection.
- **KEEP:** YOLOv8 fast primary proposals as the sole detection engine.
- **ADD:** YOLOv8 native tracking (ByteTrack) augmented with Custom Persistence Logic in Python. This fulfills the prompt's "Temporal Consistency Module" requirement with a fraction of the engineering overhead.
- **ADD:** Heavy data augmentation (Albumentations) to fulfill the "Artifact-Aware" requirement during training.

Critical Path Dependencies

1. **Colab GPU Availability:** Fine-tuning YOLOv8 on 1,612 combined images (Kvasir-SEG + CVC-ClinicDB) for 100 epochs will consume approximately 2 to 4 hours of compute time on a free-tier Google Colab T4 GPU¹². This is the primary bottleneck. Training must be initiated by Hour 12 of the hackathon at the latest to allow time for troubleshooting and inference testing.
2. **Data Preprocessing:** Neural network training cannot begin until all 1,612 binary segmentation masks are converted into YOLO .txt bounding box coordinates³⁹.

Rigorous Allocation of Roles

To maximize parallelization, the team must divide and conquer immediately upon start:

- **Team Member 1 (ML Engineer):**
 - *Hours 1-4:* Set up the Google Colab environment, execute `pip install ultralytics`, and write the Python/OpenCV script to batch-convert the downloaded masks to YOLO bounding box formats.
 - *Hours 5-12:* Configure the YOLOv8 YAML dataset files, integrate Albumentations for artifact robustness, and launch the model training on the combined static dataset⁸. Monitor loss curves.
 - *Hours 13-24:* Write the video inference script utilizing `model.track(persist=True, tracker="bytetrack.yaml")`⁵. Crucially, implement the short-term memory bounding box persistence logic (the dictionary of track IDs) to smooth the video output⁶.
- **Team Member 2 (Biomedical Engineer):**
 - *Hours 1-4:* Download Kvasir-SEG, CVC-ClinicDB, and CVC-VideoClinicDB². Curate the

validation set, ensuring that video sequences with heavy artifacts and negative sequences (no polyps, just debris) are isolated for rigorous testing.

- *Hours 5-12*: Write the mathematical formulations for the Temporal Consistency Score (TCS) and Artifact Rejection Score (ARS) into executable Python scripts, ensuring they perfectly align with clinical definitions of usability and fatigue reduction.
- *Hours 13-24*: Run the trained model against the CVC-VideoClinicDB dataset. Feed the resulting JSON/CSV coordinate outputs into the custom metric scripts to generate the final performance scores¹².
- **Team Member 3 (Generalist / Presentation Lead)**:
 - *Hours 1-24*: Build the user interface (utilizing frameworks like Streamlit or Gradio) that allows judges to ingest an MP4 video and displays a side-by-side comparison of a standard frame-wise detector (showing heavy flickering) versus the ChakraModel (showing stable, tracked bounding boxes). Construct the final pitch deck, focusing heavily on the reduction of cognitive load and the novel TCS/ARS metrics.

By leveraging highly mature open-source tooling (Ultralytics) and focusing innovation entirely on pragmatic post-processing tracking logic and custom evaluation metrics, the ChakraModel will successfully demonstrate a real-time, temporally consistent, artifact-aware detection pipeline, fulfilling all requirements by the deadline.

Works cited

1. Deep learning driven colorectal polyp analysis: a review of detection, classification and segmentation methods - Frontiers, <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1786679/full>
2. Kvasir SEG - Simula Datasets, <https://datasets.simula.no/kvasir-seg/>
3. A complete benchmark for polyp detection, segmentation and classification in colonoscopy images - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11458519/>
4. Validation of YOLOv8 algorithm in detecting colon polyps in endoscopy videos - Dao Viet, <https://imai.amegroups.org/article/view/10101/html>
5. Object Detection using yolov8 - GeeksforGeeks, <https://www.geeksforgeeks.org/machine-learning/object-detection-using-yolov8/>
6. YOLO v8n - keep original IDs of objects, once other objects are not detected, <https://stackoverflow.com/questions/76145626/yolo-v8n-keep-original-ids-of-object-s-once-other-objects-are-not-detected>
7. noushin94/EDF-YOLO-for-polyp-detection - GitHub, <https://github.com/noushin94/EDF-YOLO-for-polyp-detection>
8. Explore YOLOv8: Latest in Object Detection Tech - Viso Suite, <https://viso.ai/deep-learning/yolov8-guide/>
9. Advancing Colorectal Polyp Detection in Colonoscopy Through Region-Guided Deep Learning - MDPI, <https://www.mdpi.com/2673-4591/124/1/118>
10. (PDF) Real-Time Polyp Detection from Endoscopic Images using YOLOv8 with YOLO-Score Metrics for Enhanced Suitability Assessment - ResearchGate, https://www.researchgate.net/publication/386126737_Real-Time_Polyp_Detection

[_from_Endoscopic_Images_using_YOLOv8_with_YOLO-Score_Metrics_for_Enhanced_Suitability_Assessment](#)

11. Ultralytics/YOLOv8 - Hugging Face, <https://huggingface.co/Ultralytics/YOLOv8>
12. Object Detection using YOLOv8 - Kaggle, <https://www.kaggle.com/code/bibo600/object-detection-using-yolov8>
13. xiaochen925/AVPDN - GitHub, <https://github.com/xiaochen925/AVPDN>
14. AVPDN: learning motion-robust and scale-adaptive representations for polyp detection in dynamic colonoscopy frames - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC13057249/>
15. Polyp detection framework using Faster R-CNN object detector.... - ResearchGate, https://www.researchgate.net/figure/Polyp-detection-framework-using-Faster-R-CNN-object-detector-Pre-trained-deep-CNN_fig3_328012160
16. 3D Stable Spatio-temporal Polyp Localization in Colonoscopy Videos - UAB, <http://refbase.cvc.uab.es/files/GSF2015.pdf>
17. Deep Learning using temporal information for automatic polyp detection in videos - CEUR-WS.org, <https://ceur-ws.org/Vol-3148/paper2.pdf>
18. Code for J-BHI2023: ECC-PolypDet: Enhanced CenterNet with Contrastive Learning for Automatic Polyp Detection - GitHub, <https://github.com/yuncheng97/ECC-PolypDet>
19. Yiwen Hu - Google Scholar, <https://scholar.google.de/citations?user=dx15ucAAAAJ&hl=th>
20. Enabling Real-Time Colonoscopic Polyp Segmentation on Commodity CPUs via Ultra-Lightweight Architecture - arXiv, <https://arxiv.org/html/2602.04381v1>
21. Real-Time Polyp Detection, Localization and Segmentation in Colonoscopy Using Deep Learning - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7968127/>
22. PolypGen - Dataset Ninja, <https://datasetninja.com/polypgen>
23. Real-time deep learning-based colorectal polyp localization on clinical video footage achievable with a wide array of hardware configurations - ResearchGate, https://www.researchgate.net/publication/353142128_Real-time_deep_learning-based_colorectal_polyp_localization_on_clinical_video_footage_achievable_with_a_wide_array_of_hardware_configurations
24. A Review on Polyp Detection and Segmentation in Colonoscopy Images using Deep Learning - IJERT, <https://www.ijert.org/a-review-on-polyp-detection-and-segmentation-in-colonoscopy-images-using-deep-learning>
25. (PDF) TSdetector: Temporal-Spatial Self-correction Collaborative Learning for Colonoscopy Video Detection - ResearchGate, https://www.researchgate.net/publication/384501829_TSdetector_Temporal-Spatial_Self-correction_Collaborative_Learning_for_Colonoscopy_Video_Detection
26. ControlPolypNet: Towards Controlled Colon Polyp Synthesis for Improved Polyp Segmentation - GitHub, <https://github.com/Vanshali/ControlPolypNet>
27. Kvasir-SEG Data (Polyp segmentation & detection) - Kaggle, <https://www.kaggle.com/datasets/debeshjha1/kvasirseg>
28. The Hyper Kvasir Dataset - Kaggle,

- <https://www.kaggle.com/datasets/kelkalot/the-hyper-kvasir-dataset>
29. README.md · MedOtter/kvasir-seg at main - Hugging Face,
<https://huggingface.co/datasets/MedOtter/kvasir-seg/blob/main/README.md>
 30. CVC-ClinicDB - Dataset Ninja, <https://datasetninja.com/cvc-612>
 31. CVC-ClinicDB - Kaggle, <https://www.kaggle.com/datasets/balraj98/cvcclinicdb>
 32. Awesome-Medical-Dataset/resources/CVC-ClinicDB.md at main - GitHub,
<https://github.com/openmedlab/Awesome-Medical-Dataset/blob/main/resources/CVC-ClinicDB.md>
 33. ArdeleanRichard/YOLO-Colonoscopy: This project benchmarks modern object detection models (YOLOv8–v12, YOLO26, YOLOE, YOLO-World, RT-DETR) for automatic polyp detection. - GitHub,
<https://github.com/ArdeleanRichard/YOLO-Colonoscopy>
 34. colorectal-cancer · GitHub Topics,
<https://github.com/topics/colorectal-cancer?l=python&o=asc&s=forks>
 35. Example frames from the Kvasir dataset where we additionally have marked the polyp tissue with green outlines. - ResearchGate,
https://www.researchgate.net/figure/Example-frames-from-the-Kvasir-dataset-where-we-additionally-have-marked-the-polyp-tissue_fig1_337250576
 36. A multi-centre polyp detection and segmentation dataset for generalisability assessment, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9902556/>
 37. Real-Time Polyp Detection, Localization and Segmentation in Colonoscopy Using Deep Learning - ResearchGate,
https://www.researchgate.net/publication/349824439_Real-Time_Polyp_Detection_Localization_and_Segmentation_in_Colonoscopy_Using_Deep_Learning
 38. Colorectal Polyp Detection and Localization in Colonoscopy Images Based on YOLOv8,
https://www.researchgate.net/publication/387315944_Colorectal_Polyp_Detection_and_Localization_in_Colonoscopy_Images_Based_on_YOLOv8
 39. DepthAI Tutorial: Training and deployment of a YoloV8 model for object detection - Colab,
https://colab.research.google.com/github/luxonis/depthai-ml-training/blob/master/colab-notebooks/YoloV8_training.ipynb
 40. How to create YOLOv8-based object detection web service using Python, Julia, Node.js, JavaScript, Go and Rust - DEV Community,
<https://dev.to/andreygermanov/how-to-create-yolov8-based-object-detection-web-service-using-python-julia-nodejs-javascript-go-and-rust-4o8e>